Results of study on the Travelling salesman problem in metal hdx, as optimized/solved by nearest neighbor, 2-opt and Held-Karp.

The general scope of our project was to use the map files from the hdx metal database, adapt the metal code to be able to turn the files into arrays usable by our algorithms, and then to measure and test the nearest neighbor, 2 opt optimized nearest neighbor, and held-karp algorithms with maps with different distances between nodes, node counts, and node layouts, to figure out which optimizations/solution finders for the travelling salesman problem were usable/good with different map types.

Our main testing involved the tables below, with varying maps, having node sizes from 4 to 285 with nearest neighbor and 2-opt, and from 4 to 17, as it was enough to show how held-karp compared to the others, and held-karp became untestable on our machines past around 21~ or so, here are the tables of our main testing:

results45

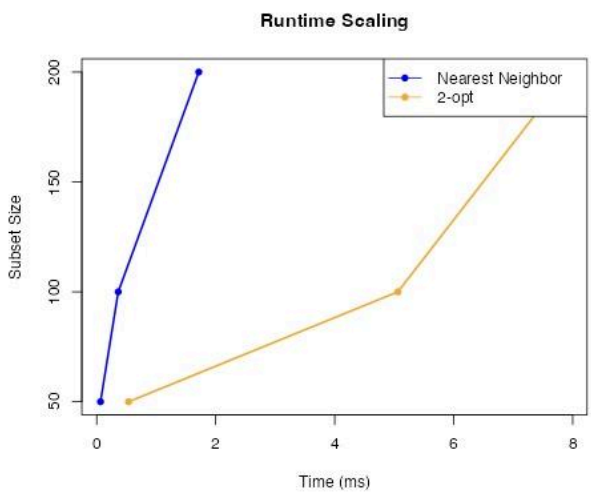
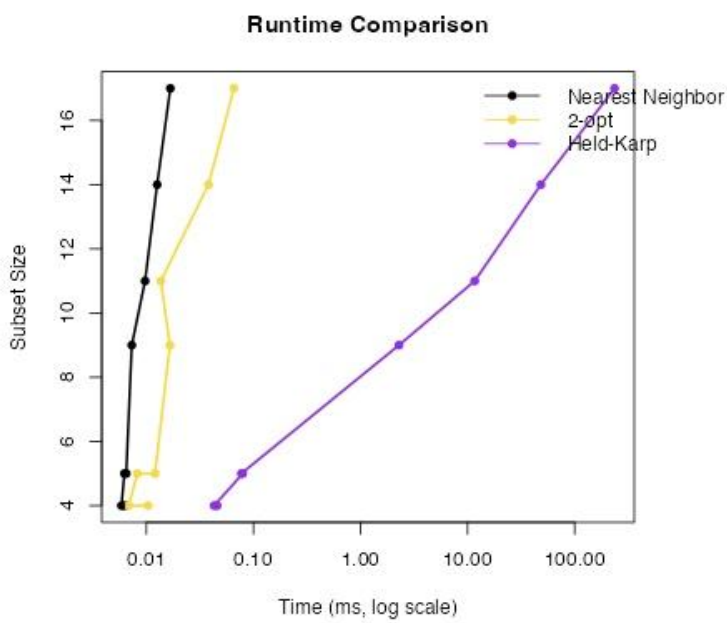
highway	subset	nn_distance	nn_time	twoopt_distance	twoopt_time	heldkarp_distance	heldkarp_time
rpi2	5	5.820	0.006232	5.820	0.008236	5.820	0.079498
amsterdam2	5	9.346	0.006482	8.128	0.012043	8.128	0.077114
BLM-region	17	13.204	0.016731	13.204	0.065713	13.204	235.931905
siena2.5	4	6.081	0.006231	6.081	0.010359	6.081	0.045766
union2	4	3.638	0.005851	3.638	0.006823	3.638	0.042940
CUW-region	11	9.249	0.009718	9.249	0.013635	9.249	11.663204
ESP-CE-region	14	16.109	0.012594	15.814	0.038001	15.814	48.409170
whitman5	9	17.378	0.007304	17.378	0.016591	17.378	2.288563

*testing was done on a separate branch that used nanotime instead of system.milliseconds

resultsNN2opt

highway	subset	nn_distance	nn_time	twoopt_distance	twoopt_time
siena2.5	4	6.081	0.006231	6.081	0.010359
union2	4	3.638	0.005851	3.638	0.006823
rpi2	5	5.820	0.006232	5.820	0.008236
amsterdam2	5	9.346	0.006482	8.128	0.012043
whitman5	9	17.378	0.007304	17.378	0.016591
CUW-region	11	9.249	0.009718	9.249	0.013635
ESP-CE-region	14	16.109	0.012594	15.814	0.038001
BRA-AM-region	16	316.022	0.013996	316.022	0.051546
BLM-region	17	13.204	0.016731	13.204	0.065713
rpi5-area	18	37.145	0.016942	27.681	0.077725
amsterdam5-area	23	59.344	0.046557	55.315	0.116698
BRA-AP-region	27	927.171	0.031147	920.810	0.140142
ABW-region	29	45.257	0.035687	41.681	0.237163
CHN-LN-region	46	940.575	0.018515	939.741	0.042028
BRA-AL-region	50	376.791	0.091711	292.881	0.854826
CHN-HN-region	50	1570.018	0.094607	1490.582	0.049011
MEX-MOR-region	50	221.414	0.084228	200.705	0.724917
MEX-NAY-region	50	513.902	0.099315	382.249	0.084147
TKM-region	50	2678.877	0.090519	2623.904	0.343962
CHN-LN-region	96	1495.306	0.088355	1405.329	0.196497
BRA-AL-region	100	879.292	0.508109	787.422	2.258828
CHN-HN-region	100	2503.166	0.330216	2393.261	0.192239
MEX-MOR-region	100	378.648	0.342840	343.307	0.271637
MEX-NAY-region	100	501.024	0.981273	493.025	0.162744
TKM-region	100	2921.534	0.347198	2861.669	1.852829
BRA-AL-region	105	938.061	0.401670	846.191	2.445286
MEX-MOR-region	144	593.988	2.730288	459.545	11.625548
TKM-region	183	3629.225	1.059459	3537.571	7.498953
CHN-HN-region	200	4244.047	0.945426	3359.157	0.981252
MEX-NAY-region	200	1195.333	0.303267	904.103	1.799660
MEX-NAY-region	237	1358.473	6.353554	1099.739	12.424189
CHN-LN-region	285	3308.783	3.863875	2991.928	21.6787

When these results were graphed and analyzed, we found that between a size of 1 and 20, held karp/2-opt only had a differnet/improved distance compared to nearest neighbor in certain cases, and only by around 6%, with the run speeds of 2-opt not being uselessly far off from nearest neighbor, while held-karp had ballooned much higher than the other 2. And from 20-285, where only nearest neighbor and 2-opt could be used, 2-opts relative runtime did increase drastically, but its distance improvement did become much better, ie going from 927->optimizing to 920 to 3300 optimizing to 2991. Overall, this was a success and a good

demonstration of how these 3 algorithms deal with the travelling salesman problem applied to a highway based map dataset.

Contributions:
Daniel:Modified HighwayGraph to work with the algorithms, implemented the algorithms to work in our code, set up driver
Ryan: Ran tests, made graphs/tables and presentation